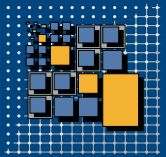
DSG-(I)DistrSys-B/M: (Introduction to) Distributed Systems

Concurrency

Summer Term 2026

Tobias Distler

University of Bamberg
Distributed Systems Group



Agenda

- Concurrency
  - Motivation
  - MapReduce
  - Threads
  - Synchronization
  - Coordination

Motivation

- Logical distribution
  - Exploiting the computational resources of **multiple processors** on a single machine
  - Multi-threaded programming

- **Concurrency**
  - ⊕ Potential to improve performance
  - ⊖ Increased complexity
  - ⊖ Need for **synchronization and coordination** among threads

- Literature

Johannes Manner and Sebastian Böhm
Lecture Notes: Concurrency Topics in Java
University of Bamberg, 2022

Agenda

- Concurrency
  - Motivation
  - MapReduce
  - Threads
  - Synchronization
  - Coordination

MapReduce

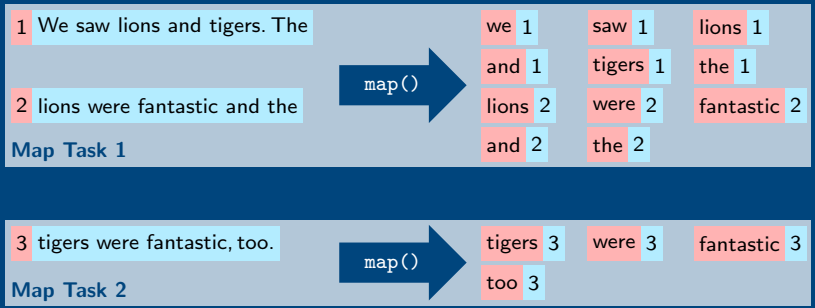
- Batch-oriented data processing
  - Indexing large data sets
  - Statistical analysis of logs
  - ...
- **Programming model**
  - Implementation of only two methods
    - `Map`: Transformation of the input data into intermediate results
    - `Reduce`: Merging of the intermediate results into the final outputs
  - Data represented as key-value pairs
- **Distributed framework**
  - Partitioning of input data
  - Dispatching and parallelized execution of tasks
- Literature

Jeffrey Dean and Sanjay Ghemawat
MapReduce: Simplified Data Processing on Large Clusters
Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI '04), 2004

MapReduce in a Nutshell

- **Task**
  - Work package processing a piece of the input/intermediate data
  - Typical configuration: # tasks $\gg$ # machines ← *Example deployment [Dean et al.]: 2.000 machines, 200.000 map tasks, 5.000 reduce tasks.*
- Main components
  - **Master** process ← *Only one in the entire system.*
  - Many **workers** processes
- Execution of a MapReduce job
  1. Master splits input into **map tasks**
  2. **Map phase:** Workers process map tasks and produce intermediate key-value pairs
  3. The framework/master...
     - ...sorts and groups all intermediate results by key and then...
     - ...splits them into **reduce tasks**
  4. **Reduce phase:** Workers process reduce tasks and produce output key-value pairs

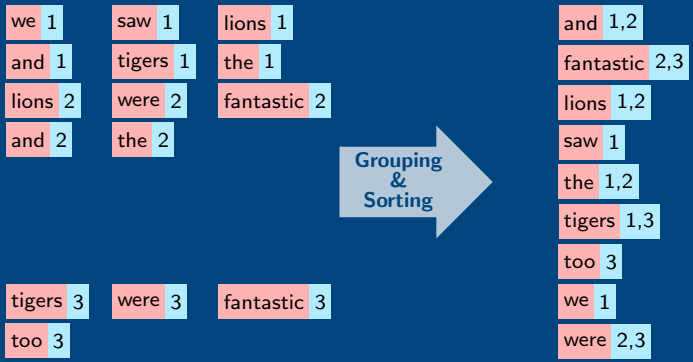
Determining the Line Number of Each Word's First Appearance with MapReduce



Assumptions: 2 map tasks, 1 reduce task, case-insensitive search.

Key Value

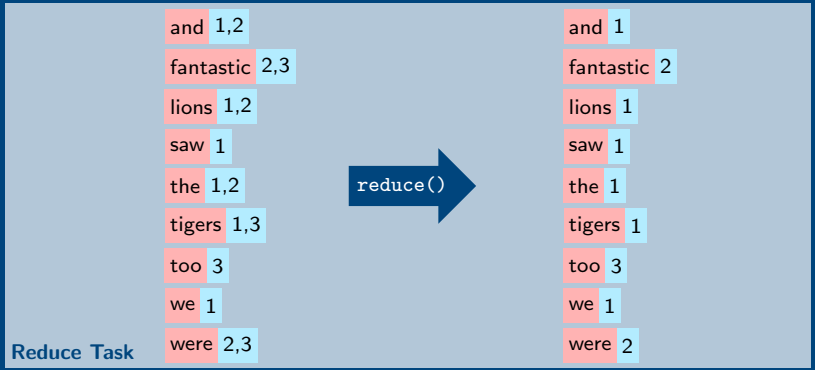
Determining the Line Number of Each Word's First Appearance with MapReduce



Assumptions: 2 map tasks, 1 reduce task, case-insensitive search.

Key | Value

Determining the Line Number of Each Word's First Appearance with MapReduce

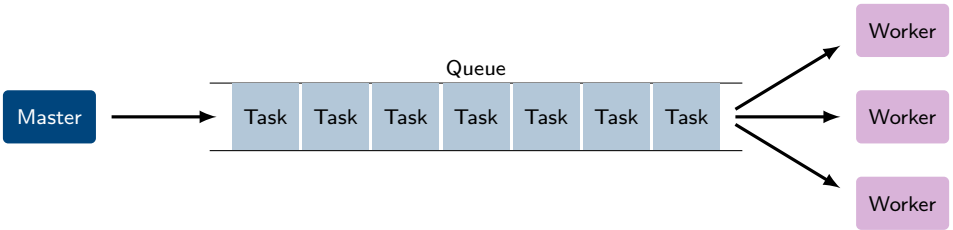


Assumptions: 2 map tasks, 1 reduce task, case-insensitive search.

Key Value

Simplified Framework for the Remainder of this Chapter

- Master and workers…
  - …**run as threads** on the same physical machine
  - …**exchange tasks via a queue** in local memory
- Java implementation
  - How to **synchronize access to the queue**?
  - How to **coordinate task handover** between master and workers?



Agenda

Threads in Java

1. Implement **custom thread class**
   - Define sub class of `java.lang.Thread`
   - Override `run()` method

```
public class DSGThread extends Thread {
  @Override
  public void run() {
    [...] // Implement the thread's execution logic here
  }
}
```

2. **Create** new thread object

```
DSGThread thread = new DSGThread();
```

3. **Start** thread by invoking the thread object's `start()` method ← *Java offers further ways to create/start a thread, which are not covered as part of the lecture.*

```
thread.start();
```

4. A thread terminates when the execution of its `run()` method finishes ← *At the end of the method, after reaching a `return` statement, or when an exception is thrown.*

Threads in Java

```java
public class DSGThreadExample {
  /* Custom thread class */
  public static class DSGThread extends Thread {
    /* Thread execution logic */
    @Override
    public void run() {
      System.out.println(
        Thread.currentThread().getName() + ": Executing"
      );
    }
  }

  /* Main method */
  public static void main(String[] args) {
    // Create thread object
    DSGThread thread = new DSGThread();
    // Wrong way to start the thread → Does not actually start a new thread
    System.out.println(
      Thread.currentThread().getName() + ": Run other thread"
    );
    thread.run();
    // Correct way to start the thread
    System.out.println(
      Thread.currentThread().getName() + ": Start other thread"
    );
    thread.start();
  }
}
```

Console Output

```
main: Run other thread
main: Executing
```

```
main: Start other thread
Thread-0: Executing
```

Preparing for Thread-Execution Interruptions in Java

- Java offers the possibility to **interrupt threads during execution** ← *The details and semantics of this mechanism are not of relevance for this lecture.*
  - The interrupted thread itself needs to handle the interrupt signal
  - Signatures of (potentially) blocking methods usually contain an `InterruptedException`

  👉 **Such exceptions must be dealt with** even if interruptions are not implemented

- Example methods of the `Thread` class

```
// Causes the currently executing thread to sleep for the specified number of milliseconds
static void sleep(long millis) throws InterruptedException
// Waits for the corresponding thread to terminate
void join() throws InterruptedException
```

- **Handling interruptions**

```
try {
  [...] // Insert the (potentially) blocking code here
} catch(InterruptedException ie) {
  [...] // Define the reaction to the interruption here
}
```

Task

```
public class DSGTask {
  /* Marker task for signaling phase completion */
  public static final DSGTask PHASE_COMPLETE = new DSGTask();
  /* Method to simulate the input-data preparation for a task */
  public void prepare() {
    delay(1);
  }
  /* Method to simulate the execution of a task */
  public void process() {
    delay(10);
  }
  /* Method to introduce artificial delays */
  private void delay(long milliseconds) {
    try {
      Thread.sleep(milliseconds);
    } catch(InterruptedException ie) {
      System.err.println("The thread was interrupted!");
    }
  }
}
```

Queue Interface

```
public interface DSGQueue {
  /* Method to append a new task to the queue */
  public void enqueue(DSGTask task);
  /* Method to remove the current head of the queue */
  public DSGTask dequeue();
}
```

Master

```
public class DSGMaster extends Thread {
  /* State */
  private final DSGQueue queue;
  /* Constructor */
  public DSGMaster(DSGQueue queue) {
    this.queue = queue;
  }
  /* Thread execution logic */
  @Override
  public void run() {
    // Create, prepare, and enqueue tasks
    for(int t = 0; t < DSGMapReduce.NUMBER_OF_TASKS; t++) {
      DSGTask task = new DSGTask();
      task.prepare();
      queue.enqueue(task);
    }
    // Signal phase completion
    for(int w = 0; w < DSGMapReduce.NUMBER_OF_WORKERS; w++) {
      queue.enqueue(DSGTask.PHASE_COMPLETE);
    }
  }
}
```

Worker

```
public class DSGWorker extends Thread {
  /* State */
  private final int id;
  private final DSGQueue queue;
  /* Constructor */
  public DSGWorker(int id, DSGQueue queue) {
    this.id = id;
    this.queue = queue;
  }
  /* Thread execution logic */
  @Override
  public void run() {
    [...]
  }
}
```

```
/* Thread execution logic */
@Override
public void run() {
  int failed = 0;
  int processed = 0;
  while(true) {
      // Try to fetch next task
      DSGTask task = queue.dequeue();
      if(task == null) {
        failed++;
        continue;
      }
      // Stop processing if there are no further tasks
      if(task == DSGTask.PHASE_COMPLETE) break;
      // Process task
      task.process();
      processed++;
  }
  System.out.println(
    id + " terminated: " +
    processed + " tasks, " +
    failed + " failed attempts"
  );
}
```

Main Program

```
public class DSGMapReduce {
  /* Constants */
  public static final int NUMBER_OF_WORKERS = 10;
  public static final int NUMBER_OF_TASKS = 100;
  /* Main */
  public static void main(String[] args) {
    // Create queue
    DSGQueue queue = new DSGSimpleQueue();
    // Create and start master
    DSGMaster master = new DSGMaster(queue);
    master.start();
    // Create and start workers
    DSGWorker[] workers = new DSGWorker[NUMBER_OF_WORKERS];
    for(int i = 0; i < workers.length; i++) {
      workers[i] = new DSGWorker(i, queue);
      workers[i].start();
    }
    try {
      // Wait for master and workers termination
      master.join();
      for(int i = 0; i < workers.length; i++) workers[i].join();
      System.out.println("All threads terminated!");
    } catch(InterruptedException ie) {
      System.err.println("Main thread interrupted!");
    }
  }
}
```

Simple Queue

```
public class DSGSimpleQueue implements DSGQueue {
  /* State */
  private final List<DSGTask> tasks;
  /* Constructor */
  public DSGSimpleQueue() {
    this.tasks = new LinkedList<DSGTask>();
  }
  /* Methods */
  @Override
  public void enqueue(DSGTask task) {
    tasks.add(task);
  }
  @Override
  public DSGTask dequeue() {
    // Return null if there is currently no task available
    if(tasks.isEmpty()) return null;
    // Remove and return next task
    DSGTask task = tasks.remove(0);
    return task;
  }
}
```

Simple Queue

```
public class DSGSimpleQueue implements DSGQueue {
  /* State */
  private final List<DSGTask> tasks;
  /* Constructor */
  public DSGSimpleQueue() {
    this.tasks = new LinkedList<DSGTask>();
  }
  /* Methods */
  @Override
  public void enqueue(DSGTask task) {
    tasks.add(task);
  }
  @Override
  public DSGTask dequeue() {
    // Return null if there is currently no task available
    if(tasks.isEmpty()) return null;
    // Remove and return next task
    DSGTask task = tasks.remove(0);
    return task;
  }
}
```

Console Output

```
Exception in thread "Thread-4" java.lang.
     IndexOutOfBoundsException: Index: 0, Size: 0
  at java.base/java.util.LinkedList.
       checkElementIndex(LinkedList.java:566)
  at java.base/java.util.LinkedList.remove(
       LinkedList.java:536)
  at DSGSimpleQueue.dequeue(DSGSimpleQueue.java:27)
  at DSGWorker.run(DSGWorker.java:24)
[Further similar exceptions]
```

Simple Queue

```
public class DSGSimpleQueue implements DSGQueue {
  /* State */
  private final List<DSGTask> tasks;
  /* Constructor */
  public DSGSimpleQueue() {
    this.tasks = new LinkedList<DSGTask>();
  }
  /* Methods */
  @Override
  public void enqueue(DSGTask task) {
    tasks.add(task);
  }
  @Override
  public DSGTask dequeue() {
    // Return null if there is currently no task available
    if(tasks.isEmpty()) return null;
    // Remove and return next task
    DSGTask task = tasks.remove(0);
    return task;
  }
}
```

Console Output

```
Exception in thread "Thread-4" java.lang.
     IndexOutOfBoundsException: Index: 0, Size: -3
  at java.base/java.util.LinkedList.
       checkElementIndex(LinkedList.java:566)
  at java.base/java.util.LinkedList.remove(
       LinkedList.java:536)
  at DSGSimpleQueue.dequeue(DSGSimpleQueue.java:27)
  at DSGWorker.run(DSGWorker.java:24)
[Further similar exceptions]
```

Implementation is **not thread-safe!**

Agenda

- Concurrency
  - Motivation
  - MapReduce
  - Threads
  - Synchronization
  - Coordination

Critical Sections

- General concept
  - Means to achieve **mutual exclusion** in a program
  - Only **at most a single thread** is allowed to enter a specific critical section at a time
  - Other threads are suspended if they also request access to the same section
  - Multiple independent critical sections may exist within the same program

- Java specifics
  - Definition of blocks using the `synchronized` keyword
  - Use of **synchronization objects**
    - Each block must be mapped to exactly one object
    - All blocks referring to **the same object belong to the same critical section**
    - The keyword `this` denotes the current object

Think of the synchronization object as a name of the critical section.

```
synchronized([Reference to synchronization object]) {
  [...]  // Code inside the critical section
}
```

Synchronized Queue

```
public class DSGSynchronizedQueue implements DSGQueue {
  /* State */
  private final List<DSGTask> tasks;
  /* Constructor */
  public DSGSynchronizedQueue() {
    this.tasks = new LinkedList<DSGTask>();
  }
  /* Methods */
  @Override
  public void enqueue(DSGTask task) {
    synchronized(this) {
      tasks.add(task);
    }
  }
  @Override
  public DSGTask dequeue() {
    DSGTask task;
    synchronized(this) {
      // Return null if there is currently no task available
      if(tasks.isEmpty()) return null;
      // Remove next task
      task = tasks.remove(0);
    }
    return task;
  }
}
```

Synchronized Queue

```
public class DSGSynchronizedQueue implements DSGQueue {
  /* State */
  private final List<DSGTask> tasks;
  /* Constructor */
  public DSGSynchronizedQueue() {
    this.tasks = new LinkedList<DSGTask>();
  }
  /* Methods */
  @Override
  public void enqueue(DSGTask task) {
    synchronized(this) {
      tasks.add(task);
    }
  }
  @Override
  public DSGTask dequeue() {
    DSGTask task;
    synchronized(this) {
      // Return null if there is currently no task available
      if(tasks.isEmpty()) return null;
      // Remove next task
      task = tasks.remove(0);
    }
    return task;
  }
}
```

Console Output

```
9 terminated: 5 tasks, 395826 failed attempts
1 terminated: 17 tasks, 329225 failed attempts
5 terminated: 4 tasks, 418126 failed attempts
3 terminated: 9 tasks, 438762 failed attempts
4 terminated: 16 tasks, 304570 failed attempts
8 terminated: 13 tasks, 322434 failed attempts
6 terminated: 6 tasks, 299330 failed attempts
2 terminated: 9 tasks, 351512 failed attempts
0 terminated: 4 tasks, 348266 failed attempts
7 terminated: 17 tasks, 347956 failed attempts
All threads terminated!
```

Locks

- General concept
  - Each lock represents a separate critical section
  - Only **at most one thread is able to hold a lock** at a time
  - Dedicated **methods for entering and leaving** the critical section

Offers more flexibility than `synchronized` blocks, but also entails greater risk of making mistakes.

- Java specifics

Package: `java.util.concurrent.locks`

  - Interface: `Lock`
  - Implementation: `ReentrantLock`

```
public interface Lock {
  // Acquires the lock
  void lock();
  // Releases the lock
  void unlock();
  [Further methods]
}
```

Lock-based Queue

```
public class DSGLockQueue implements DSGQueue {
  /* State */
  private final List<DSGTask> tasks;
  private final Lock critical;
  /* Constructor */
  public DSGLockQueue() {
    this.tasks = new LinkedList<DSGTask>();
    this.critical = new ReentrantLock();
  }
  /* Methods */
  public void enqueue(DSGTask task) {
    critical.lock();
    tasks.add(task);
    critical.unlock();
  }
  public DSGTask dequeue() {
    // Enter critical section
    critical.lock();
    // Return null if there is currently no task available
    if(tasks.isEmpty()) {
      critical.unlock(); // Leave critical section
      return null;
    }
    // Remove next task
    DSGTask task = tasks.remove(0);
    // Leave critical section
    critical.unlock();
    return task;
  }
}
```

Lock-based Queue

```
public class DSGLockQueue implements DSGQueue {
  /* State */
  private final List<DSGTask> tasks;
  private final Lock critical;

  /* Constructor */
  public DSGLockQueue() {
    this.tasks = new LinkedList<DSGTask>();
    this.critical = new ReentrantLock();
  }

  /* Methods */
  public void enqueue(DSGTask task) {
    critical.lock();
    tasks.add(task);
    critical.unlock();
  }

  public DSGTask dequeue() {
    // Enter critical section
    critical.lock();
    // Return null if there is currently no task available
    if(tasks.isEmpty()) {
      critical.unlock(); // Leave critical section
      return null;
    }
    // Remove next task
    DSGTask task = tasks.remove(0);
    // Leave critical section
    critical.unlock();
    return task;
  }
}
```

Console Output

```
4 terminated: 9 tasks, 785752 failed attempts
9 terminated: 8 tasks, 1074772 failed attempts
7 terminated: 10 tasks, 953694 failed attempts
3 terminated: 11 tasks, 937443 failed attempts
0 terminated: 7 tasks, 909678 failed attempts
1 terminated: 9 tasks, 1001105 failed attempts
6 terminated: 3 tasks, 1052420 failed attempts
5 terminated: 13 tasks, 995627 failed attempts
8 terminated: 19 tasks, 835273 failed attempts
2 terminated: 11 tasks, 906473 failed attempts
All threads terminated!
```

Lock-based Queue

```
public class DSGLockQueue implements DSGQueue {
  /* State */
  private final List<DSGTask> tasks;
  private final Lock critical;
  /* Constructor */
  public DSGLockQueue() {
    this.tasks = new LinkedList<DSGTask>();
    this.critical = new ReentrantLock();
  }
  /* Methods */
  public void enqueue(DSGTask task) {
    critical.lock();
    tasks.add(task);
    critical.unlock();
  }
  public DSGTask dequeue() {
    // Enter critical section
    critical.lock();
    // Return null if there is currently no task available
    if(tasks.isEmpty()) {
      critical.unlock(); // Leave critical section
      return null;
    }
    // Remove next task
    DSGTask task = tasks.remove(0);
    // Leave critical section
    critical.unlock();
    return task;
  }
}
```

Console Output

```
4 terminated: 9 tasks, 785752 failed attempts
9 terminated: 8 tasks, 1074772 failed attempts
7 terminated: 10 tasks, 953694 failed attempts
3 terminated: 11 tasks, 937443 failed attempts
0 terminated: 7 tasks, 909678 failed attempts
1 terminated: 9 tasks, 1001105 failed attempts
6 terminated: 3 tasks, 1052420 failed attempts
5 terminated: 13 tasks, 995627 failed attempts
8 terminated: 19 tasks, 835273 failed attempts
2 terminated: 11 tasks, 906473 failed attempts
All threads terminated!
```

Many **failed dequeue()** attempts!

Agenda

- Concurrency
  - Motivation
  - MapReduce
  - Threads
  - Synchronization
  - Coordination

Coordination

- Problem
  - A thread (*consumer*) **requires input from another thread** (*producer*) to make progress
  - Polling the producer for input is often inefficient ← *This approach is also known as "busy waiting".*
- Solution: Mechanism for **signaling that a certain condition is fulfilled**
  - Consumer blocks only if the condition is not yet met
  - Producer signals condition
  - Consumer unblocks on signal
- Example: **Dedicated object for coordination**

```
public interface DSGCoordinator {
  /* Method to signal that a new task is available */
  public void produce();
  /* Method to wait until a new task becomes available */
  public void consume();
}
```

Master

```
public class DSGMaster extends Thread {
  /* State */
  private final DSGQueue queue;
  private final DSGCoordinator coordinator;
  /* Constructor */
  public DSGMaster(DSGQueue queue, DSGCoordinator coordinator) {
    this.queue = queue;
    this.coordinator = coordinator;
  }
  /* Thread execution logic */
  @Override
  public void run() {
    // Create, prepare, and enqueue tasks
    for(int t = 0; t < DSGMapReduce.NUMBER_OF_TASKS; t++) {
      DSGTask task = new DSGTask();
      task.prepare();
      queue.enqueue(task);
      coordinator.produce();
    }
    // Signal phase completion
    for(int w = 0; w < DSGMapReduce.NUMBER_OF_WORKERS; w++) {
      queue.enqueue(DSGTask.PHASE_COMPLETE);
      coordinator.produce();
    }
  }
}
```

Worker

```java
public class DSGWorker extends Thread {
  /* State */
  private final int id;
  private final DSGQueue queue;
  private final DSGCoordinator coordinator;
  /* Constructor */
  public DSGWorker(int id, DSGQueue queue,
                   DSGCoordinator coordinator) {
    this.id = id;
    this.queue = queue;
    this.coordinator = coordinator;
  }
  /* Thread execution logic */
  @Override
  public void run() {
    [...]
  }
}
```

```java
/* Thread execution logic */
@Override
public void run() {
  int failed = 0;
  int processed = 0;
  while(true) {
      // Try to fetch next task
      coordinator.consume();
      DSGTask task = queue.dequeue();
      if(task == null) {
        failed++;
        continue;
      }
      // Stop processing if there are no further tasks
      if(task == DSGTask.PHASE_COMPLETE) break;
      // Process task
      task.process();
      processed++;
  }
  System.out.println(
    id + " terminated: " +
    processed + " tasks, " +
    failed + " failed attempts"
  );
}
```

Main Program

```
public class DSGMapReduce {
  /* Constants */
  public static final int NUMBER_OF_WORKERS = 10;
  public static final int NUMBER_OF_TASKS = 100;
  /* Main */
  public static void main(String[] args) {
    // Create queue and coordinator
    DSGQueue queue = new DSGSynchronizedQueue();
    DSGCoordinator coordinator = new DSGSimpleCoordinator();
    // Create and start master
    DSGMaster master = new DSGMaster(queue, coordinator);
    master.start();
    // Create and start workers
    DSGWorker[] workers = new DSGWorker[NUMBER_OF_WORKERS];
    for(int i = 0; i < workers.length; i++) {
      workers[i] = new DSGWorker(i, queue, coordinator);
      workers[i].start();
    }
    try {
      // Wait for master and workers termination
      master.join();
      for(int i = 0; i < workers.length; i++) workers[i].join();
      System.out.println("All threads terminated!");
    } catch(InterruptedException ie) {
      System.err.println("Main thread interrupted!");
    }
  }
}
```

Coordination in Java

- General concept
  - Use of **synchronization object as meeting point** for consumer(s) and producer(s)
  - Threads must **enter the corresponding critical section** before invoking the related methods
- Consumer
  - **Potentially blocking** method

```
// Causes the current thread to wait until it is awakened
void wait() throws InterruptedException
```

  - Releases the critical section (temporarily) ← *This is necessary, because otherwise the producer would not be able to enter.*
- Producer
  - **Non-blocking** methods

```
// Wakes up a single waiting thread
void notify()
// Wakes up all waiting threads
void notifyAll()
```

  - Programmer has **no control over thread selection** or the order in which threads resume execution

Simple Coordinator

```
public class DSGSimpleCoordinator implements DSGCoordinator {
  @Override
  public void produce() {
    synchronized(this) {
      notify();
    }
  }
  @Override
  public void consume() {
    try {
      synchronized(this) {
        wait();
      }
    } catch(InterruptedException ie) {
      System.err.println("The thread was interrupted!");
    }
  }
}
```

Simple Coordinator

```
public class DSGSimpleCoordinator implements DSGCoordinator {
  @Override
  public void produce() {
    synchronized(this) {
      notify();
    }
  }
  @Override
  public void consume() {
    try {
      synchronized(this) {
        wait();
      }
    } catch(InterruptedException ie) {
      System.err.println("The thread was interrupted!");
    }
  }
}
```

Console Output

```
9 terminated: 5 tasks, 69 failed attempts
3 terminated: 3 tasks, 81 failed attempts
5 terminated: 8 tasks, 56 failed attempts
7 terminated: 10 tasks, 43 failed attempts
```

[Program does not terminate...]

Simple Coordinator

```
public class DSGSimpleCoordinator implements DSGCoordinator {
  @Override
  public void produce() {
    synchronized(this) {
      notify();
    }
  }
  @Override
  public void consume() {
    try {
      synchronized(this) {
        wait();
      }
    } catch(InterruptedException ie) {
      System.err.println("The thread was interrupted!");
    }
  }
}
```

Console Output

```
9 terminated: 5 tasks, 69 failed attempts
3 terminated: 3 tasks, 81 failed attempts
5 terminated: 8 tasks, 56 failed attempts
7 terminated: 10 tasks, 43 failed attempts
```

[Program does not terminate...]

Implementation **loses events!**

Lost Wake-Ups

- **Problem** in `DSGSimpleCoordinator`
  - Java does not buffer notifications
  - Invocations of `notify()` / `notifyAll()` **only have an effect if at least one thread is waiting**

  👉 Lost wake-ups if `produce()` is called before `consume()`

- Possible solutions
  - Ensure that `produce()` is always called after `consume()` ← *This approach is not really practical.*
  - **Keep track of notifications** using a counter
    - Initial counter value is zero
    - Increase counter on each `produce()`
    - Only block `consume()` if the counter is currently at zero
    - Decrease counter on each `consume()`

    👉 Semantics of a **semaphore** ← *As implemented in `java.util.concurrent.Semaphore`.*

Counting Coordinator

```
public class DSGCountingCoordinator implements DSGCoordinator {
  /* State */
  private int counter;
  /* Constructor */
  public DSGCountingCoordinator() {
    this.counter = 0;
  }
  /* Methods */
  @Override
  public void produce() {
    synchronized(this) {
      counter++;
      notify();
    }
  }
  @Override
  public void consume() {
    synchronized(this) {
      while(counter <= 0) {
        try {
          wait();
        } catch(InterruptedException ie) {
          System.err.println("The thread was interrupted!");
          return;
        }
      }
      counter--;
    }
  }
}
```

Counting Coordinator

```java
public class DSGCountingCoordinator implements DSGCoordinator {
  /* State */
  private int counter;
  /* Constructor */
  public DSGCountingCoordinator() {
    this.counter = 0;
  }
  /* Methods */
  @Override
  public void produce() {
    synchronized(this) {
      counter++;
      notify();
    }
  }
  @Override
  public void consume() {
    synchronized(this) {
      while(counter <= 0) {
        try {
          wait();
        } catch(InterruptedException ie) {
          System.err.println("The thread was interrupted!");
          return;
        }
      }
      counter--;
    }
  }
}
```

Console Output

```
3 terminated: 9 tasks, 0 failed attempts
7 terminated: 3 tasks, 0 failed attempts
1 terminated: 15 tasks, 0 failed attempts
6 terminated: 7 tasks, 0 failed attempts
0 terminated: 14 tasks, 0 failed attempts
5 terminated: 11 tasks, 0 failed attempts
4 terminated: 15 tasks, 0 failed attempts
2 terminated: 6 tasks, 0 failed attempts
9 terminated: 7 tasks, 0 failed attempts
8 terminated: 13 tasks, 0 failed attempts
All threads terminated!
```

Semaphore-based Coordinator

```
public class DSGSemaphoreCoordinator implements DSGCoordinator {
  /* State */
  private final Semaphore semaphore;
  /* Constructor */
  public DSGSemaphoreCoordinator() {
    this.semaphore = new Semaphore(0);
  }
  /* Methods */
  @Override
  public void produce() {
    semaphore.release();
  }
  @Override
  public void consume() {
    semaphore.acquireUninterruptibly();
  }
}
```

Semaphore-based Coordinator

```
public class DSGSemaphoreCoordinator implements DSGCoordinator {
  /* State */
  private final Semaphore semaphore;
  /* Constructor */
  public DSGSemaphoreCoordinator() {
    this.semaphore = new Semaphore(0);
  }
  /* Methods */
  @Override
  public void produce() {
    semaphore.release();
  }
  @Override
  public void consume() {
    semaphore.acquireUninterruptibly();
  }
}
```

Console Output

```
3 terminated: 9 tasks, 0 failed attempts
4 terminated: 8 tasks, 0 failed attempts
9 terminated: 10 tasks, 0 failed attempts
0 terminated: 9 tasks, 0 failed attempts
2 terminated: 11 tasks, 0 failed attempts
7 terminated: 9 tasks, 0 failed attempts
8 terminated: 7 tasks, 0 failed attempts
1 terminated: 15 tasks, 0 failed attempts
5 terminated: 12 tasks, 0 failed attempts
6 terminated: 10 tasks, 0 failed attempts
All threads terminated!
```

Concluding Remark

- The coordinator functionality is typically **not implemented in a separate object**

- Improvement
  - **Integrate coordinator with the queue**
  - Block `dequeue()` call if the queue is currently empty
  - Signal the availability of new elements inside `enqueue()`

Implement this solution as part of Task 1 of your assignment